

EXTERNAL API · V1.5

API

EDESIS External API

Entegrasyon Rehberi — Tam Referans

Öğrenci · Sınav · Sonuç · Analiz · Karne · Sonuç Gönderimi

Sürüm: v1.5

Tarih: 4 Temmuz 2026

Doğrulama: Kaynak kod ile birebir · canlı tenant testi

İçindekiler

1. Genel Bakış
2. Kimlik Doğrulama (Authentication)
3. Yetkilendirme: Scope'lar ve Paketler
 - Paketler
 - Endpoint → scope eşlemesi
4. Rate Limiting
5. Hata Yönetimi
6. Sayfalama (Pagination)
7. Okuma Endpoint'leri (GET)
 - 7.1 Öğrenciler
 - 7.2 Dönemler
 - 7.3 Sınıf Seviyeleri ve Şubeler
 - 7.4 Sınavlar
 - 7.4.1 Sınav Yapısı (ingest için) — v1.5
 - 7.5 Sınav Sonuçları
 - 7.6 Sonuç Kırılımları (Ders ve Konu)
 - 7.6.1 Gönderim Değerlendirme Durumu (ingest) — v1.5
 - 7.7 Ödevler
 - 7.8 Ders Programı
 - 7.9 Kitaplar
 - 7.10 Analiz Raporları
 - 7.11 Rehberlik Görüşmeleri
8. Yazma Endpoint'leri (POST)
 - 8.1 Şube Oluşturma
 - 8.2 Öğrenci Ekleme
 - 8.3 Veli Ekleme
 - 8.4 Ham Sınav Cevabı Gönderimi (Ingest) — v1.5
9. Karne / Rapor Oluşturma
10. Webhook: App.Exam.ResultsPublished
11. Entegrasyon Rehberi
 - Veri okuma akışı
 - Artımlı senkron akışı (önerilen)
 - Veri yazma akışı (admin paketi gerekli)
 - Sınav sonucu gönderme akışı (ingest — v1.5, exam_results:write gerekli)
12. Kod Örnekleri
 - cURL
 - C# (HttpClient)
 - Python (requests)

Ek A: KVKK ve Veri Koruma Notları

Ek B: Deęişiklik Geçmişı

Sürüm: v1.5 · **Tarih:** 2026-07-04 · Bu doküman v1.1 PDF rehberi + v1.3/v1.4/v1.5 eklemelerinin birleştirilmiş tam halidir; tüm endpoint'ler kaynak kod ile birebir doğrulanmıştır (canlı doğrulama: v1.4 → 2026-07-01 demo tenant; v1.5 ingest uçları → 2026-07-04 canlı, bursa tenant).

v1.5 yenilikleri (özet) — Sınav Sonucu Gönderimi (ham cevap ingest): Kurumlar artık ham optik cevapları API üzerinden gönderip Edesis'e değerlendirebilir. Üç yeni uç: `GET /exams/{examId}/structure` (ne göndereceğini öğren), `POST /exams/{examId}/results` (ham cevapları gönder → **202** + `jobId` + `statusUrl`), `GET /exams/{examId}/results/status` (asenكرون değerlendirme izle). Puanlama **server tarafında**, UI optik import'u ile **birebir aynı** job ile yapılır. Yeni scope: `exam_results:write` (yalnız admin paketi veya custom key). Detay: **§7.4.1, §7.5.1, §8.4, §11 (ingest akışı)**.

v1.4 yenilikleri (özet): `GET /students` artık sunucu tarafında `TermId`, `StudentState`, `ClassroomId`, `IsActive` ve `ModifiedAfter` filtrelerini uygular (önceden bu parametreler sessizce yok sayılıyor, her zaman tüm liste dönüyordu). Yanıta `termId`, `termName`, `studentState`, `classroomId`, `modifiedDate` alanları eklendi; `totalCount` artık filtrelenmiş toplamı döndürür. Detay: **§7.1**.

1. Genel Bakış

Edesis External API, kurumların kendi sistemlerine öğrenci verileri, sınav sonuçları, analiz raporları ve karneleri entegre etmesini sağlar. Veriler ağırlıklı **read-only** sunulur; istisnalar: öğrenci/veli/şube ekleme (admin paketi), karne oluşturma ve **ham sınav cevabı gönderimi** (v1.5 — admin paketi/ `exam_results:write`).

Özellik	Detay
Base URL	<code>https://{domain}/api/external/v1</code>
Protokol	HTTPS (zorunlu)
Format	JSON (UTF-8)
Kimlik doğrulama	API Key (<code>X-API-Key</code> header)
Rate limit	standard: 60 istek/dk · premium: 300 istek/dk
Veri izolasyonu	Her API key yalnızca kendi kurumunun verilerine erişir

Veri güvenliği: API key'iniz yalnızca size ait kurumun verilerine erişim sağlar. Başka kurumların verilerine erişim teknik olarak mümkün değildir (tenant izolasyonu sunucu tarafında zorunlu uygulanır).

2. Kimlik Doğrulama (Authentication)

Tüm isteklerde `X-API-Key` header'ı zorunludur. API key'inizi kurum yöneticinizden alabilirsiniz.

Key formatı: `edesis_{kurum}_{ortam}_{id}_{rastgele}` — ör.

`edesis_demo_live_xK3m_7Kx9mQ2pR4sT6uV8wY...`

```
GET /api/external/v1/students HTTP/1.1
Host: api.edesis.com
X-API-Key: edesis_demo_live_xK3m_7Kx9mQ2pR4sT6uV8wY...
```

Hata yanıtları:

```
// Key gönderilmedi → 401
{ "error": "X-API-Key header is required for external API access" }

// Geçersiz format veya eşleşmeyen key → 401
{ "error": "Invalid API Key format" }
{ "error": "Invalid API Key" }

// Pasifleştirilmiş key → 403
{ "error": "API Key is deactivated" }

// Süresi dolmuş key → 403
{ "error": "API Key has expired" }

// IP kısıtlı key, izinsiz IP → 403
{ "error": "IP address not allowed for this API Key" }

// Scope yetersiz → 403
{ "error": "Insufficient scope. Required: 'exam_results:read'" }
```

API key güvenliği: Key oluşturulduktan sonra tekrar gösterilemez (sunucuda yalnızca hash saklanır). Kaybolursa yeni key oluşturulması gerekir. Key'lere son kullanma tarihi (`expiresAt`) ve IP kısıtlaması (virgülle ayrılmış IP listesi) tanımlanabilir.

3. Yetkilendirme: Scope'lar ve Paketler

Her endpoint bir scope gerektirir; key'in scope'ları seçilen pakete göre belirlenir. `custom` pakette scope'lar key bazında tek tek atanır.

Paketler

Paket	Erişim kapsamı
<code>basic</code>	Öğrenciler, sınıflar, sınıf seviyeleri, dönemler (read-only)
<code>exams</code>	basic + sınavlar, sonuçlar, analiz (read-only)
<code>attendance</code>	basic + yoklama (read-only)
<code>finance</code>	Faturalar, ödemeler (read-only)
<code>student_dashboard</code>	Sınav, ödev, program, kitap, analiz, rehberlik, karne (read-only + rapor)
<code>full_read</code>	Tüm veriler (read-only) + karne oluşturma
<code>admin</code>	Tüm okuma + öğrenci/veli/şube ekleme + ham sınav sonucu gönderimi (<code>exam_results:write</code>)
<code>custom</code>	Key bazında serbest scope ataması (ör. yalnız <code>exam_results:write</code>)

Endpoint → scope eşlemesi

Endpoint	Scope
GET /students , GET /students/{id}	students:read
GET /students içinde tcNo alanı	students:read_pii (ek olarak)
POST /students	students:write
POST /parents	parents:write
GET /classrooms	classrooms:read
POST /classrooms	classrooms:write
GET /grades	grades:read
GET /terms	terms:read
GET /exams , GET /exams/{id}/subjects , GET /exams/{id}/structure	exams:read
GET /exams/{id}/results , GET /exams/results , GET /exams/{id}/results/lessons , GET /exams/{id}/results/subjects , GET /exams/{id}/results/status	exam_results:read
POST /exams/{id}/results (ham cevap gönderimi)	exam_results:write (v1.5)
GET /analytics/reports , GET /analytics/reports/student/{id}	analytics:read
GET /homework , GET /homework/student/{id}	homework:read
GET /schedule	schedule:read
GET /books	books:read
GET /guidance/{studentId}	guidance:read
POST /reports/exam-report , GET /reports/job-status/{jobId}	reports:generate

students:read_pii (TC Kimlik No): Hiçbir hazır pakete dahil değildir; yalnızca **custom** paketle key bazında atanır. KVKK gereği sözleşmesel onay şartına bağlıdır.

exam_results:write (v1.5 — ham cevap gönderimi): Yalnızca **admin** paketinde veya **custom** pakette key bazında bulunur. Salt-okuma (**exam_results:read**) key ile **POST /exams/{id}/results** çağrısı **403** döner. Okuma uçları (**/structure** , **/status**) için bu write scope'u **gerekmez** — **exams:read** / **exam_results:read** yeterlidir.

Fail-closed yetkilendirme (v1.5): Middleware artık fail-closed çalışır — bir scope'a eşlenmemiş external endpoint erişimi **403** ile reddedilir (önceden read-key ile istenmeyen yazma yolu oluşabiliyordu; kapatıldı).

4. Rate Limiting

Limitler **dakika bazlı kayan pencere** ile uygulanır:

Tier	İstek/dakika
standard	60
premium	300

Her başarılı yanıtta `X-RateLimit-Limit` header'ı döner. Limit aşımında:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 10
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 0

{ "error": "Rate limit exceeded. Retry after 10 seconds." }
```


5. Hata Yönetimi

HTTP kodu	Anlam	Aksiyon
200	Başarılı	Yanıtı işleyin
204	Kayıt bulunamadı (tekil GET)	ID'yi kontrol edin
400	Geçersiz istek (gövde/model binding)	İstek gövdesini kontrol edin
401	Kimlik doğrulama hatası	API key'i kontrol edin
403	Yetki hatası (scope/IP/pasif/süresi dolmuş key)	Hata mesajındaki nedeni kontrol edin
404	Kaynak yok / başka tenant (ör. yabancı examId)	ID'yi ve tenant'ı kontrol edin
409	Çakışma — mevcut sonuç var (POST /results , replace=false)	replace=true ile gönderin
422	İş kuralı reddi — hiçbir satır kabul edilmedi (POST /results)	rejected[] nedenlerini düzeltin
429	Rate limit aşımı	Retry-After kadar bekleyin
500	Sunucu hatası	Destek ile iletişime geçin

İş kuralı hataları (ör. eksik zorunlu alan) `UserFriendlyException` olarak anlamlı Türkçe mesajla döner.

6. Sayfalama (Pagination)

Liste endpoint'leri ortak parametreleri destekler:

Parametre	Tip	Varsayılan	Açıklama
MaxResultCount	integer	10	Sayfa başına kayıt (kırılım uçlarında max 100)
SkipCount	integer	0	Atlanacak kayıt sayısı
Filter	string	—	Metin araması (endpoint'e göre ad/başlık alanlarında)
Sorting	string	—	Sıralama (destekleyen uçlarda, ör. "name asc")

Sayfalı yanıt yapısı:

```
{ "totalCount": 6747, "items": [ { }, { } ] }
```

GET /terms , GET /grades , GET /exams/{id}/subjects sayfalama olmadan düz dizi döndürür.

7. Okuma Endpoint'leri (GET)

7.1 Öğrenciler

GET /students — scope: **students:read** · Kurumdaki öğrencileri listeler (sayfalı). Sayfalama (**SkipCount** / **MaxResultCount**) ve **Filter** (ad/soyad araması) dışında aşağıdaki filtreler **sunucu tarafında** uygulanır (hepsi opsiyonel, geriye uyumlu):

Ek parametre	Tip	Açıklama
TermId	integer	Yalnızca bu döneme kayıtlı öğrenciler (GET /terms id değeri)
StudentState	string (enum)	Kayıt durumu filtresi — geçerli değerler aşağıda
ClassroomId	integer	Şube filtresi (GET /classrooms id değeri)
IsActive	boolean	true : durumu boş veya Kesinkayıt olanlar; false : diğerleri
ModifiedAfter	ISO-8601 datetime (UTC)	Artımlı senkron: bu tarihten sonra değişen kayıtlar

v1.4 öncesi davranış: Bu parametreler tanınmıyor ve **sessizce yok sayılıyordu** (her istek tüm listeyi döndürüyordu). v1.4 ile filtreler etkin; **totalCount** artık **filtrelenmiş** toplamı döndürür.

StudentState geçerli değerleri (büyük/küçük harf duyarlı): **Misafir**, **Aday**, **Onkayıt**, **Kesinkayıt**, **KayıtSildirdi**, **Uzaklastirildi**, **Mezun**, **SinavSonucuIcin**, **OzelDers**, **DenemeKulubu**, **KutupHane**. Geçersiz / sayısal / virgülle birden fazla değer gönderilirse **400** + geçerli değer listesi döner (stack trace sızmaz):

```
// GET /students?StudentState=Donduruldu → 400
{ "message": "Geçersiz StudentState değeri. Geçerli değerler: Misafir, Aday, Onkayıt, Kesinkayıt, ..." }
```

GET /students/{id} — scope: **students:read** · Tek öğrenci döndürür (bulunamazsa **204** boş yanıt).

```
{
  "id": 7203743,
  "firstName": "SELEN",
  "lastName": "BAYKAL",
  "studentNumber": "20259",
  "tcNoHash": "a3f1...64-karakter-hex", // SHA-256(tcNo) – TCNo kayıtlıysa her zaman dolu
  "tcNo": null, // yalnız students:read_pii scope'u ile dolu
  "className": "12-A", // "gradeName-şubeadı" formatı (= classroomName)
  "gradeName": "12",
  "isActive": true,
  "email": "ornek@mail.com",
  "phone": "5551234567",
  "termId": 40, // v1.4 – öğrencinin dönem ID'si
  "termName": "2024-2025", // v1.4 – dönem adı
  "studentState": "Kesinkayit", // v1.4 – kayıt durumu (yukarıdaki enum)
  "classroomId": 456, // v1.4 – şube ID'si
  "modifiedDate": "2026-04-11T14:32:00" // v1.4 – son değişiklik anı (ModifiedAfter ile eşleşir)
}
```

- **tcNoHash** eşleştirme amaçlıdır; **tuzsuz SHA-256** olduğundan kişisel veri gibi korunmalıdır.
- Öğrencinin TC kimlik numarası sistemde kayıtlı değilse **tcNoHash** da **null** döner.
- Eşleştirme öncelik önerisi: **studentNumber** → **tcNoHash** → **phone** → manuel. **studentNumber** kurum içinde **unique garanti edilmez**.
- **className** zaten **"12-A"** (gradeName-şubeadı) formatındadır ve talep edilen **classroomName** değerine karşılık gelir; ayrı bir alan eklenmemiştir.
- **Artımlı öğrenci senkronu**: son başarılı senkron zamanını saklayıp **?ModifiedAfter={sonSenkron}** ile poll edin; **modifiedDate** alanını yeni işaretleyici olarak güncelleyin.

7.2 Dönemler

GET /terms — scope: **terms:read** · Akademik dönem listesi (düz dizi). **isDefault: true** olan aktif dönemdir. Host + kendi kurumunuza ait dönemleri döndürür.

Doğru path: **GET /api/external/v1/terms** 'tir. (**../api/services/app/External/Terms** **değildir**.)
Dönen **id** değerleri **GET /students?TermId=...** filtresinde kullanılır (§7.1).

```
[
  { "id": 98, "name": "2025-2026", "isDefault": true },
  { "id": 40, "name": "2024-2025", "isDefault": false }
]
```

7.3 Sınıf Seviyeleri ve Şubeler

GET /grades — scope: **grades:read** · Sınıf seviyelerini listeler (düz dizi). Şube oluşturmada **gradeId** olarak kullanılır.

```
[ { "id": 5, "name": "9. Sınıf" }, { "id": 8, "name": "12. Sınıf" } ]
```

GET /classrooms — scope: **classrooms:read** · Kurumdaki şubeleri listeler (sayfalı).

```
{ "totalCount": 24, "items": [
  { "id": 456, "name": "A", "gradeName": "12. Sınıf", "fullName": "12. Sınıf-A" } ] }
```

7.4 Sınavlar

GET /exams — scope: **exams:read** · Sınav listesi (sayfalı, tarihe göre azalan).

Ek parametre	Açıklama
Filter	Sınav adında arama
resultsUpdatedAfter	Sonuçları bu andan sonra oluşan/güncellenen sınavlar (artımlı senkron)

```
{
  "id": 1264223,
  "name": "TYT Deneme 5",
  "examDate": "2026-04-10T00:00:00",
  "examType": "TYT Genel", // sınav türü
  "totalQuestions": 120, // deneme cevap anahtarındaki toplam soru
  "studentCount": 245, // sınava giren öğrenci sayısı
  "createdAt": "2026-04-01T09:12:00",
  "resultsUpdatedAt": "2026-04-11T14:32:00", // sonuçların son oluşma/güncellenme anı
  "resultStatus": "Ready", // "None" | "Processing" | "Ready"
  "subjectNos": [1101, 1102, 1203] // sınavın konu kodu (KonuKodu) listesi
}
```

- resultStatus** : **None** = sonuç yok, **Processing** = değerlendirme sürüyor, **Ready** = hazır.
- Artımlı senkron önerisi**: son başarılı senkron zamanını saklayıp **?resultsUpdatedAfter=...** ile poll edin.

GET /exams/{examId}/subjects — scope: **exams:read** · Sınavın konu listesi (düz dizi). All Star tarzı **subjectNos** eşleştirmesi için konu detayları:

```
[{ "konuId": 4512, "konuKodu": 1101, "konuAdi": "Sözcükte Anlam",
  "lessonId": 3, "lessonName": "Türkçe", "questionCount": 6 }]
```

konuKodu null olan cevap anahtarları satırları **/exams** yanıtındaki **subjectNos** dizisinde yer almaz; bu endpoint'te **konuKodu: null** olarak dönebilir.

7.4.1 Sınav Yapısı (ingest için) — v1.5

GET /exams/{examId}/subjects 'ten farkı: **subjects** konu (**KonuKodu**) kırılımını, **structure** ise **ders/kitapçık** kırılımını verir. Ham cevap göndermeden önce (§8.4) her ders için **kaç harf** gönderileceğini ve hangi **lessonId** / **dersGrupId** / **kitapcikTuru** kombinasyonlarının beklendiğini bu uçtan öğrenin.

GET /exams/{examId}/structure — scope: **exams:read** · Sınavın cevap anahtarı yapısını (düz dizi) döndürür. Her satır bir **kitapçık x ders** kombinasyonudur.

```
[
  { "kitapcikTuru": "A", "lessonId": 1, "lessonName": "Matematik", "dersGrupId": 1,
    "questionCount": 40 },
  { "kitapcikTuru": "A", "lessonId": 2, "lessonName": "Türkçe", "dersGrupId": 6,
    "questionCount": 40 },
  { "kitapcikTuru": "A", "lessonId": 3, "lessonName": "Tarih", "dersGrupId": 7,
    "questionCount": 5 },
  { "kitapcikTuru": "A", "lessonId": 8, "lessonName": "Fizik", "dersGrupId": 2,
    "questionCount": 7 }
]
```

Alan	Açıklama
<code>kitapcikTuru</code>	Kitapçık türü (ör. "A", "B"). Gönderimde satır (öğrenci) seviyesinde belirtilir.
<code>lessonId</code>	Ders ID — gönderimde <code>dersCevapları[].lessonId</code> olarak birebir kullanılır
<code>lessonName</code>	Ders adı (bilgi amaçlı)
<code>dersGrupId</code>	Ders grubu ID — gönderimde <code>dersCevapları[].dersGrupId</code> olarak kullanılır
<code>questionCount</code>	Bu ders için beklenen cevap uzunluğu — <code>cevaplar</code> string'i tam bu uzunlukta olmalı

Doğrulama: POST `/results` gönderiminde her ders için `cevaplar` uzunluğu buradaki `questionCount` ile **birebir** eşleşmeli; aksi halde satır reddedilir (§8.4). Sınavın tüm dersleri gönderilmelidir. Aynı `examId` birden fazla kitapçık türü içeriyorsa her öğrenci kendi kitapçık türünü (`kitapcikTuru`) belirtir.

7.5 Sınav Sonuçları

GET `/exams/{examId}/results` — scope: `exam_results:read` · Belirli sınavın tüm öğrenci sonuçları (sayfalı, ada göre sıralı). **Filter**: öğrenci ad/soyad araması.

Bu uç `StudentId` parametresi **desteklemez**; öğrenci bazlı sorgu için aşağıdaki `/exams/results` ucunu veya kırılım uçlarını kullanın.

GET `/exams/results` — scope: `exam_results:read` · Filtrelenmiş sınav sonuçları (sayfalı, sınav tarihine göre azalan).

Ek parametre	Açıklama
<code>examId</code>	Sınav bazlı filtre
<code>studentId</code>	Öğrenci bazlı filtre
<code>startDate</code> , <code>endDate</code>	Sınav tarihi aralığı
<code>updatedAfter</code>	Satır bazında artımlı senkron (sonucu bu andan sonra değişenler)
Filter	Öğrenci ad/soyad araması

```
{
  "studentId": 7203743,
  "studentName": "BAYKAL SELEN",
  "studentNumber": "20259",
  "className": "12-A",
  "examId": 1264223,
  "examName": "TYT Deneme 5",
  "examDate": "2026-04-10T00:00:00",
  "score": 30.75,
  "net": 30.75, // score ile aynı kaynak: toplam net
  "correctCount": 45,
  "wrongCount": 15,
  "emptyCount": 60,
  "totalQuestions": 120, // öğrencinin cevap kağıdındaki toplam (C+W+E)
  "successRate": 37.5, // correct*100/totalQuestions, 2 hane
  "updatedAt": "2026-04-11T14:32:00"
}
```

7.6 Sonuç Kırılımları (Ders ve Konu)

Her iki uçta sayfalama: varsayılan 50, **max 100** kayıt/sayfa.

GET /exams/{examId}/results/lessons — scope: **exam_results:read** · Öğrenci × ders (test) kırılımı.
studentId (opsiyonel) ile tek öğrenciye daraltılır.

```
{ "studentId": 7203743, "studentNumber": "20259", "lessonId": 3, "lessonName": "Türkçe",
  "correctCount": 28, "wrongCount": 8, "emptyCount": 4, "net": 26.0 }
```

GET /exams/{examId}/results/subjects — scope: **exam_results:read** · Öğrenci × konu kırılımı.
studentId (opsiyonel).

```
{ "studentId": 7203743, "studentNumber": "20259", "konuId": 4512, "konuKodu": 1101,
  "konuAdi": "Sözcükte Anlam", "lessonId": 3, "lessonName": "Türkçe",
  "correctCount": 4, "wrongCount": 1, "emptyCount": 1 }
```

Büyük sınavlarda konu kırılımının **studentId** ile öğrenci bazlı çekilmesi önerilir.

7.6.1 Gönderim Değerlendirme Durumu (ingest) — v1.5

GET /exams/{examId}/results/status?jobId={jobId} — scope: **exam_results:read** · **POST /exams/{examId}/results** çağrısının döndürdüğü **jobId** ile asenkron değerlendirmenin durumunu sorgular.

```
{ "jobId": "a1b2c3d4-...", "state": "Completed", "message": "Değerlendirme tamamlandı." }
```

state değerleri: **Pending** (kuyrukta) · **Running** (değerlendiriliyor) · **Completed** (bitti — puanlanmış sonuçlar
GET /exams/{id}/results 'tan çekilebilir) · **Failed** (hata).

Geçersiz / süresi dolmuş jobId : Uç **404 değil, 200** döner ve { "state": "NotFound", "message": "İş bulunamadı (süresi dolmuş veya geçersiz jobId)." } gövdesi gelir. Yani **state** alanı fiilen **NotFound** değerini de alabilir; istemci bunu **Failed** 'den ayrı ele almalıdır. (Sadece **başka tenant'ın / olmayan examId** 'si 404 verir.)

7.7 Ödevler

GET /homework — scope: **homework:read** · Kurumdaki ödevleri listeler (sayfalı, teslim tarihine göre azalan).

GET /homework/student/{studentId} — scope: **homework:read** · Öğrencinin ödevleri ve tamamlanma durumları.

Ek parametre	Açıklama
Filter	Ödev başlığında arama
OnlyPending	true : yalnız tamamlanmamış ödevler
DonemId	Dönem filtresi

```
{
  "id": 496758,
  "title": "15. Hafta Matematik",
  "description": "Fonksiyonlar konu tekrarı",
  "startDate": "2026-04-09T00:00:00",
  "dueDate": "2026-04-25T00:00:00",
  "isCompleted": false,
  "completedDate": null
}
```

7.8 Ders Programı

GET /schedule — scope: **schedule:read** · Ders programını listeler (sayfalı).

Ek parametre	Açıklama
StudentId	Öğrenci bazlı filtre
StartDate , EndDate	Tarih aralığı

```
{
  "id": 21943260,
  "startDate": "2026-04-10T08:00:00",
  "endDate": "2026-04-10T08:40:00",
  "lessonName": "Matematik",
  "teacherName": "AHMET YILMAZ",
  "className": "12-A",
  "description": null,
  "isCancelled": false,
  "lessonNo": 1
}
```


7.9 Kitaplar

GET /books — scope: **books:read** · Öğrenci kitap atamalarını listeler (sayfalı). **studentId** (opsiyonel) ile öğrenci bazlı filtre.

```
{ "id": 2964720, "bookName": "TYT Matematik Soru Bankası",  
  "studentId": 7105077, "studentName": "ARDA YİĞİT", "className": "12-A" }
```

7.10 Analiz Raporları

GET /analytics/reports — scope: **analytics:read** · Tüm analiz raporlarını listeler (sayfalı).

GET /analytics/reports/student/{studentId} — scope: **analytics:read** · Öğrencinin analiz raporları.

```
{ "id": 88412, "fileName": "analiz-raporu.pdf", "reportType": "OncelikliKonu",  
  "studentId": 7105077, "studentName": "ARDA YİĞİT", "className": "12-A",  
  "analysisId": 512, "analysisName": "Nisan Analizi", "creationTime":  
  "2026-04-12T10:00:00" }
```

7.11 Rehberlik Görüşmeleri

GET /guidance/{studentId} — scope: **guidance:read** · Öğrencinin **onaylanmış** rehberlik görüşme özetlerini listeler (sayfalı, tarihe göre azalan). Veliyle paylaşılabılır özet (**message**) döner.

```
{ "id": 3321, "date": "2026-04-08T00:00:00", "message": "Sınav kaygısı üzerine  
görüşüldü...",  
  "teacherName": "AYŞE DEMİR", "studentName": "ARDA YİĞİT",  
  "category": "Akademik", "isApproved": true }
```

8. Yazma Endpoint'leri (POST)

Yazma yetkileri: Yazma uçları ayrı scope gerektirir (`students:write` , `parents:write` , `classrooms:write`). Read scope'ları ile yazma yapılamaz. Yazma yetkisi için **admin** paketi (veya custom pakette ilgili write scope'ları) gereklidir.

8.1 Şube Oluşturma

POST /classrooms — scope: `classrooms:write` · Yeni şube oluşturur. Aynı sınıf seviyesinde aynı isimle şube varsa hata döner.

```
// İstek
{
  "name": "A",           // Zorunlu — şube adı (max 20 karakter)
  "gradeId": 8,          // Zorunlu — sınıf seviyesi ID (GET /grades'den alın)
  "donemId": 98,         // Opsiyonel — dönem ID
  "capacity": 30         // Opsiyonel — kapasite (0-200)
}

// Yanıt
{ "id": 456, "name": "A", "gradeName": "12. Sınıf", "fullName": "12. Sınıf-A" }
```

8.2 Öğrenci Ekleme

POST /students — scope: `students:write` · Yeni öğrenci ve kullanıcı hesabı oluşturur; öğrenci sisteme bu hespla giriş yapabilir.

Öğrenci kotası: Kurumunuzun öğrenci kotası dolmuşsa yeni öğrenci eklenemez.

```
// İstek
{
  "firstName": "ARDA",           // Zorunlu
  "lastName": "YİĞİT",          // Zorunlu
  "email": "arda@mail.com",      // Zorunlu — giriş için kullanılır
  "password": "Sifre123!",       // Zorunlu — ilk giriş şifresi
  "phone": "5551234567",        // Opsiyonel
  "classroomId": 456,           // Opsiyonel — şube ID
  "donemId": 98,                 // Opsiyonel — dönem ID
  "studentNumber": 20259,        // Opsiyonel — okul numarası
  "tcNo": 12345678901,          // Opsiyonel — TC (11 hane, sayısal)
  "birthDate": "2008-05-15",    // Opsiyonel
  "gender": "Erkek",             // Opsiyonel — Erkek / Kadın
}

// Yanıt
{ "id": 7205001, "userId": 98456, "firstName": "ARDA", "lastName": "YİĞİT",
  "userName": "arda_x4k2", "className": "12. Sınıf-A" }
```

8.3 Veli Ekleme

POST /parents — scope: **parents:write** · Öğrenciye bağlı veli hesabı oluşturur; veli sisteme giriş yapıp öğrencisini takip edebilir.

```
// İstek
{
  "studentId": 7205001,           // Zorunlu — hangi öğrencinin velisi
  "firstName": "MEHMET",         // Zorunlu
  "lastName": "YİĞİT",           // Zorunlu
  "email": "mehmet@mail.com",    // Zorunlu
  "password": "Sifre123!",        // Zorunlu
  "phone": "5559876543",         // Opsiyonel
  "relationship": "Baba",         // Anne / Baba / Diğer
  "tcNo": "12345678901",         // Opsiyonel
  "isPaymentResponsible": true    // Ödeme sorumlusu mu
}

// Yanıt
{ "id": 12345, "userId": 98457, "studentId": 7205001,
  "firstName": "MEHMET", "lastName": "YİĞİT" }
```

8.4 Ham Sınav Cevabı Gönderimi (Ingest) — v1.5

Yazma yetkisi: **exam_results:write** scope'u gerekir (admin paketi veya custom key). Salt-okuma key ile **403** döner. Puanlama **server tarafında** yapılır — kurum yalnızca ham cevap gönderir; Edesis mevcut cevap anahtarına göre **asen kron** değerlendirir (UI'daki optik import ile birebir aynı değerlendirme job'u).

POST /exams/{examId}/results — scope: **exam_results:write** · Öğrencilerin ham optik cevaplarını gönderir. Gönderim kabul edilirse **202 Accepted** + **jobId** döner; değerlendirme arka planda çalışır, durum §7.6.1'den izlenir.

⚠ **replace** bir GÖVDE alanıdır, query parametresi DEĞİLDİR. Kök seviyede **"replace": true** gönderin. **?replace=true** query string olarak gönderilirse sessizce yok sayılır ve mevcut sonuç varsa **409** alırsınız (runtime doğrulandı: 2026-07-04).

Gövde alanı	Tip	Varsayılan	Açıklama
replace	boolean	false	true : bu sınav için öğrencinin mevcut sonuçlarını sil + yeniden değerlendir (sonuç çoğalmaz , üzerine yazılır). false /gönderilmez + mevcut sonuç → 409 .

İstek gövdesi:

```
// POST /api/external/v1/exams/1508004/results
{
  "replace": false, // GÖVDEDE – true: mevcut sonucun üzerine yaz
  "results": [ // Zorunlu – en az 1 öğrenci satırı
    {
      // Öğrenci kimliği – biri zorunlu. Öncelik: ogrenciId > okulNumarasi > tcNo
      "ogrenciId": 972746, // Opsiyonel – Edesis öğrenci ID (integer)
      "okulNumarasi": 1000, // Opsiyonel – okul numarası (SAYISAL, string değil)
      "tcNo": 12345678901, // Opsiyonel – TC kimlik no (11 hane, SAYISAL)
      "kitapcikTuru": "A", // Zorunlu – kitapçık türü (öğrenci/satır seviyesinde)
      "dersCevaplari": [ // Zorunlu – sınavın TÜM dersleri gönderilmeli
        { "lessonId": 1, "dersGrupId": 1, "cevaplar": "ABCDEABCDE...ABCD" }, // 40 harf
        { "lessonId": 2, "dersGrupId": 6, "cevaplar": "BADCE...E" } // 40 harf
        // ... GET /structure'daki her (lessonId, dersGrupId) için bir satır
      ]
    }
  ]
}
```

Alanlar:

Alan	Tip	Zorunlu	Açıklama
<code>replace</code>	boolean	—	Kök seviyede. <code>true</code> : mevcut sonucun üzerine yaz. Varsayılan <code>false</code> .
<code>results[]</code>	dizi	✓	En az bir öğrenci sonucu. Boşsa/eksikse 400 .
<code>ogrenciId</code> / <code>okulNumarasi</code> / <code>tcNo</code>	integer	biri ✓	Öğrenci eşleştirme. Sayısal ; <code>okulNumarasi</code> / <code>tcNo</code> string gönderilirse 400 (binding).
<code>kitapcikTuru</code>	string	✓	<code>GET /structure</code> 'daki değerlerden biri (ör. <code>"A"</code>). Satır seviyesinde.
<code>dersCevaplari[]</code>	dizi	✓	Sınavın tüm dersleri. <code>lessonId</code> + <code>dersGrupId</code> + <code>cevaplar</code> .
<code>cevaplar</code>	string	✓	Ham cevap dizisi; uzunluğu <code>structure.questionCount</code> ile birebir eşleşmeli.

Yanıt — kabul edildi (202):

```
// HTTP 202 Accepted
{
  "accepted": 245, // değerlendirilmeye alınan öğrenci sayısı
  "rejected": [ // eşleşmeyen/hatalı satırlar (varsa)
    { "index": 12, "reason": "OkulNumarasi ile öğrenci bulunamadı." }
  ],
  "jobId": "a1b2c3d4-...", // değerlendirme job ID'si
  "statusUrl": "/api/external/v1/exams/1508004/results/status?jobId=a1b2c3d4-...",
  "message": "Değerlendirme başlatıldı."
}
```

Kısmi kabul: Geçerli satırlar `accepted` sayılır ve değerlendirilir; her reddedilen satır `rejected[]` 'e düşer (`index` = gönderilen dizideki sıra, `reason` = neden). Reddedilen satır için hiçbir şey yazılmaz.

Yanıt — hiçbir satır kabul edilmedi (422):

```
// HTTP 422 Unprocessable Entity – geçerli satır yok, jobId üretilmez
{ "accepted": 0,
  "rejected": [ { "index": 0, "reason": "Cevap uzunluğu cevap anahtarı ile eşleşmiyor.
LessonId=1, DersGrupId=1, Beklenen=40, Gelen=2." } ],
  "jobId": null, "statusUrl": null, "message": "Kabul edilen öğrenci sonucu yok." }
```

Reddetme (`rejected[].reason`) nedenleri — runtime doğrulandı:

Durum	reason
Kimlik alanı yok	<code>OgrenciId</code> , <code>OkulNumarasi</code> veya <code>TcNo</code> alanlarından biri zorunludur.
Öğrenci bulunamadı	<code>OgrenciId</code> ile öğrenci bulunamadı. (ilgili alana göre değişir)
Ders listesi boş	<code>DersCevapları</code> zorunludur.
Cevap uzunluğu yanlış	<code>Cevap uzunluğu cevap anahtarı ile eşleşmiyor. LessonId=.., Beklenen=.., Gelen=..</code>
Geçersiz kitapçık	<code>Kitapçık türü için cevap anahtarı bulunamadı. KitapcikTuru=..</code>
Geçersiz ders/grup	<code>Cevap anahtarı bulunamadı. KitapcikTuru=.., LessonId=.., DersGrupId=..</code>

Eşleştirme kuralı: Öncelik `ogrenciId` > `okulNumarasi` > `tcNo`. Eşleşmeyen veya birden fazla öğrenciye eşleşen satır reddedilir; otomatik öğrenci oluşturulmaz.

HTTP durum kodları (`POST /results`):

Kod	Anlam
202	En az bir satır kabul edildi; <code>jobId</code> üretildi (bazı satırlar <code>rejected[]</code> 'de olabilir)
400	Gövde/model hatası (ör. <code>results</code> boş, <code>okulNumarasi</code> string)
403	<code>exam_results:write</code> scope'u yok
404	<code>examId</code> yok veya başka tenant'a ait
409	<code>replace=false</code> ve öğrencinin bu sınavda mevcut sonucu var
422	Hiçbir satır kabul edilmedi (tüm satırlar <code>rejected[]</code>)

Bilinen sınır (pre-existing): `DersSoruNumarasi` 1'den başlamayan dersler (ör. seçmeli ders 21–25 arası numaralı) değerlendirme job'unda puanlanmıyor (0/0/0). Bu, external ingest'e özgü değildir — UI optik import ile **birebir aynı** davranır. Sınav yapısında böyle ders varsa entegrasyon öncesi teyit edin.

9. Karne / Rapor Oluşturma

Yazma işlemi: Bu endpoint PDF rapor üretir ve sunucu kaynağı tüketir. Gereksiz tekrar çağrılardan kaçının
— `forceNew: false` ile mevcut rapor varsa yeniden oluşturulmaz.

POST /reports/exam-report — scope: `reports:generate` · Sınav karnesi/raporu oluşturur ve PDF URL döndürür.

```
// İstek
{
  "examId": 1264223,           // Zorunlu — sınav ID
  "termId": 40,               // Zorunlu — dönem ID
  "studentIds": [7203743],    // Zorunlu — öğrenci ID'leri (en az 1)
  "reportCodes": [102],       // Opsiyonel — rapor türü (varsayılan: 102 = Karne)
  "forceNew": false           // true: yeniden oluştur; false: varsa mevcudu döndür
}
```

Rapor türleri:

Kod	Tür	Açıklama
102	Karne	Öğrenci sınav karnesi
104	BK-5 Analiz	Son 5 sınav karşılaştırma
105	BK-10 Analiz	Son 10 sınav karşılaştırma

```
// Yanıt — hazır (küçük öğrenci grupları senkron üretilir)
{ "status": "Completed", "reportUrl": "https://cdn.edesis.com/rapor/.../karne.pdf",
  "jobId": null, "pollUrl": null, "message": "Rapor hazır." }

// Yanıt — işleniyor (büyük gruplar arka planda üretilir)
{ "status": "Processing", "reportUrl": null, "jobId": "a1b2c3d4-...",
  "pollUrl": "/api/external/v1/reports/job-status/a1b2c3d4-...",
  "message": "Rapor arka planda oluşturuluyor. PollUrl ile durumu sorgulayın." }
```

GET /reports/job-status/{jobId} — scope: `reports:generate` · Arka planda oluşturulan raporun durumunu sorgular; `status: "Completed"` olduğunda yanıtaki rapor URL alanından PDF indirilebilir.

10. Webhook: `App.Exam.ResultsPublished`

Sınav değerlendirme (optik aktarım sonrası) tamamlandığında tetiklenir. Abonelik, kurum yönetim paneli veya WebhookSubscription API'si ile yapılır (URL + secret; ABP imza header'ları ve otomatik retry geçerlidir).

Payload (`event.data`):

```
{
  "provider": "EDESIS",
  "examId": 1264223,
  "examName": "TYT Deneme 5",
  "examDate": "2026-04-10T00:00:00",
  "examType": "TYT Genel",
  "totalQuestions": 120,
  "studentCount": 245,
  "resultsUpdatedAt": "2026-04-11T14:32:00",
  "resultsUrl": "/api/external/v1/exams/1264223/results"
}
```

- Sonuç listesi payload'a **gömülmez**; `resultsUrl` 'den sayfa çekilir (büyük sınavlarda payload boyutu ve retry maliyeti nedeniyle).
- Aynı sınavın optiği yeniden yüklenirse event **yeniden** tetiklenir; tüketici `examId` ile upsert (idempotent) çalışmalı, `resultsUpdatedAt` ayırt edicidir.

11. Entegrasyon Rehberi

Veri okuma akışı

1. **API key alın** — kurum yöneticinizden talep edin.
2. **Öğrenci listesini çekin** — `GET /students` ; `id` alanını sisteminize kaydedin, diğer uçlarda bu ID kullanılır.
3. **Dönem ve sınav bilgilerini alın** — `GET /terms` , `GET /exams` .
4. **Sınav sonuçlarını sorgulayın** — `GET /exams/results?studentId={id}` (öğrenci bazlı) veya `GET /exams/{examId}/results` (sınav bazlı).
5. **Kırılım gerekiyorsa** — `GET /exams/{examId}/results/lessons` ve `/results/subjects` .
6. **Karne oluşturun** — `POST /reports/exam-report` ; yanıtta `reportUrl` ile PDF'i indirin.

Artımlı senkron akışı (önerilen)

1. Webhook aboneliği yapın (`App.Exam.ResultsPublished`) **veya** periyodik olarak `GET /exams?resultsUpdatedAfter={sonSenkron}` ile poll edin.
2. Değişen sınavlar için `GET /exams/{examId}/results` (veya satır bazında `GET /exams/results?updatedAfter=...`) ile sonuçları çekin.
3. `examId` + `studentId` anahtarıyla **upsert** yapın; `resultsUpdatedAt` / `updatedAt` alanlarını son senkron zamanı olarak saklayın.

Veri yazma akışı (admin paketi gerekli)

1. **Sınıf seviyelerini alın** — `GET /grades` (şube oluşturmak için `gradeId` gereklidir).
2. **Şube oluşturun** — `POST /classrooms` ; yanıtta `id` öğrenci atamasında kullanılır.
3. **Öğrenci ekleyin** — `POST /students` (`classroomId` ile şubeye atayın); yanıtta `id` ve `userId` değerlerini kaydedin.
4. **Veli ekleyin** — `POST /parents` (`studentId` önceki adımdan gelir).

Senkronizasyon önerisi: Öğrenci listesini günde 1 kez, sınav sonuçlarını sınav sonrası (webhook veya `resultsUpdatedAfter` ile) çekin. Gereksiz sık çağrılardan kaçının — rate limit'e takılabilirsiniz.

Sınav sonucu gönderme akışı (ingest — v1.5, `exam_results:write` gerekli)

Kurum kendi optik okuyucusundan gelen ham cevapları Edesis'e değerlendirtir:

1. **Yapıyı öğren** — `GET /exams/{examId}/structure` ; her ders için beklenen `lessonId` / `dersGrupId` / `kitapcikTuru` / `questionCount` (cevap uzunluğu) bilgisini alın.
2. **Öğrencileri eşleştir** — mümkünse `ogrenciId` (en güvenilir), yoksa `okulNumarasi` veya `tcNo` . `ogrenciId` için `GET /students` 'ten `id` alanını kullanın.
3. **Ham cevapları gönder** — `POST /exams/{examId}/results` ; her öğrenci için sınavın **tüm** derslerini, her `cevaplar` string'i `questionCount` uzunluğunda olacak şekilde. Yanıtta `rejected[]` listesini loglayın (eşleşmeyen öğrenci vb.).

4. **Değerlendirmeyi izle** — `GET /exams/{examId}/results/status?jobId={jobId}; state == "Completed"` olana kadar makul aralıkla (ör. 5–10 sn) poll edin.
5. **Puanlanmış sonuçları çek** — `GET /exams/{examId}/results` (ve gerekirse `/results/lessons` , `/results/subjects`). Puanlama server tarafında yapıldığından doğru/yanlış/net değerleri cevap anahtarıyla tutarlıdır.

Tekrar gönderim: Aynı öğrenci için sonuç varken yeniden göndermek isterseniz gövdede `"replace": true` gönderin (mevcut silinir + yeniden değerlendirilir; sonuç çoğalmaz). `replace` gövde alanıdır — query string (`?replace=true`) **yok sayılır**. `replace=false` (varsayılan) + mevcut sonuç → **409**.

12. Kod Örnekleri

cURL

```

# Öğrenci listesi
curl -s "https://api.edesis.com/api/external/v1/students?MaxResultCount=10" \
  -H "X-API-Key: YOUR_API_KEY"

# Öğrenci listesi - dönem + kayıt durumu filtresi (v1.4)
curl -s "https://api.edesis.com/api/external/v1/students?
TermId=40&StudentState=Kesinkayit&MaxResultCount=500" \
  -H "X-API-Key: YOUR_API_KEY"

# Öğrenci listesi - şube filtresi (v1.4)
curl -s "https://api.edesis.com/api/external/v1/students?ClassroomId=456" \
  -H "X-API-Key: YOUR_API_KEY"

# Öğrenci listesi - artımlı senkron (v1.4)
curl -s "https://api.edesis.com/api/external/v1/students?
ModifiedAfter=2025-01-01T00:00:00Z" \
  -H "X-API-Key: YOUR_API_KEY"

# Sınav sonuçları (öğrenci bazlı, artımlı)
curl -s "https://api.edesis.com/api/external/v1/exams/results?
studentId=7105077&updatedAfter=2026-04-10T00:00:00" \
  -H "X-API-Key: YOUR_API_KEY"

# Sınavın konu listesi
curl -s "https://api.edesis.com/api/external/v1/exams/1264223/subjects" \
  -H "X-API-Key: YOUR_API_KEY"

# Sınav yapısı - ingest için beklenen ders/kitapçık/soru sayısı (v1.5)
curl -s "https://api.edesis.com/api/external/v1/exams/1264223/structure" \
  -H "X-API-Key: YOUR_API_KEY"

# Ham cevap gönderimi - ingest (v1.5, exam_results:write) → 202 + jobId
# NOT: replace GÖVDEDE gönderilir (query ?replace=true YOK SAYILIR)
curl -s "https://api.edesis.com/api/external/v1/exams/1264223/results" \
  -X POST \
  -H "X-API-Key: YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"replace":false,"results":[{"okuNumarasi":1000,"kitapcikTuru":"A","dersCevaplari":
[{"lessonId":1,"dersGrupId":1,"cevaplar":"ABCDE...(40 harf)"},
{"lessonId":2,"dersGrupId":6,"cevaplar":"BADCE...(40 harf)"}]}]}'

# Değerlendirme durumu (yukarıdaki yanıttan gelen jobId ile) (v1.5)
curl -s "https://api.edesis.com/api/external/v1/exams/1264223/results/status?jobId=JOB_ID" \
  -H "X-API-Key: YOUR_API_KEY"

# Konu kırılımı (öğrenci bazlı)
curl -s "https://api.edesis.com/api/external/v1/exams/1264223/results/subjects?
studentId=7203743" \
  -H "X-API-Key: YOUR_API_KEY"

# Karne oluştur
curl -s "https://api.edesis.com/api/external/v1/reports/exam-report" \
  -X POST \
  -H "X-API-Key: YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"examId":1264223,"termId":40,"studentIds":[7203743],"reportCodes":[102]}'

```

```
# Şube oluştur (admin paketi)
curl -s "https://api.edesis.com/api/external/v1/classrooms" \
  -X POST \
  -H "X-API-Key: YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"name":"A","gradeId":8,"donemId":98}'

# Öğrenci ekle (admin paketi)
curl -s "https://api.edesis.com/api/external/v1/students" \
  -X POST \
  -H "X-API-Key: YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"firstName":"ARDA","lastName":"YİĞİT","email":"arda@mail.com","password":"Sifre123!","classroomId":456}'

# Veli ekle (admin paketi)
curl -s "https://api.edesis.com/api/external/v1/parents" \
  -X POST \
  -H "X-API-Key: YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"studentId":7205001,"firstName":"MEHMET","lastName":"YİĞİT","email":"mehmet@mail.com","password":"Sifre123!"}'
```

C# (HttpClient)

```
using var client = new HttpClient();
client.BaseAddress = new Uri("https://api.edesis.com");
client.DefaultRequestHeaders.Add("X-API-Key", "YOUR_API_KEY");

// Öğrenci listesi
var response = await client.GetAsync("/api/external/v1/students?MaxResultCount=100");
var json = await response.Content.ReadAsStringAsync();
var result = JsonSerializer.Deserialize<PagedResult<StudentDto>>(json);

foreach (var student in result.Items)
    Console.WriteLine($"{student.Id} - {student.FirstName} {student.LastName}");

// Karne oluştur
var body = JsonSerializer.Serialize(new
{
    examId = 1264223,
    termId = 40,
    studentIds = new[] { 7203743 },
    reportCodes = new[] { 102 },
    forceNew = false
});
var content = new StringContent(body, Encoding.UTF8, "application/json");
var reportResponse = await client.PostAsync("/api/external/v1/reports/exam-report",
content);
var reportJson = await reportResponse.Content.ReadAsStringAsync();
// reportJson.reportUrl → PDF indirme linki
```

Python (requests)

```

import requests

BASE_URL = "https://api.edesis.com/api/external/v1"
HEADERS = {"X-API-Key": "YOUR_API_KEY"}

# Öğrenci listesi
r = requests.get(f"{BASE_URL}/students", headers=HEADERS,
                 params={"MaxResultCount": 100})
students = r.json()["items"]
for s in students:
    print(f"{s['id']} - {s['firstName']} {s['lastName']}")

# Artımlı sınav senkronu
r = requests.get(f"{BASE_URL}/exams", headers=HEADERS,
                 params={"resultsUpdatedAfter": "2026-04-10T00:00:00"})
for exam in r.json()["items"]:
    res = requests.get(f"{BASE_URL}/exams/{exam['id']}/results", headers=HEADERS,
                      params={"MaxResultCount": 100})
    # examId + studentId ile upsert edin

# Karne oluşturma
r = requests.post(f"{BASE_URL}/reports/exam-report", headers=HEADERS,
                 json={"examId": 1264223, "termId": 40,
                      "studentIds": [7203743], "reportCodes": [102]})
result = r.json()
if result["status"] == "Completed":
    print(f"PDF: {result['reportUrl']}")

# Ham cevap gönderimi (ingest) + değerlendirmeyi izleme (v1.5)
import time

exam_id = 1264223
# 1) Yapıyı öğren (her ders için beklenen cevap uzunluğu)
structure = requests.get(f"{BASE_URL}/exams/{exam_id}/structure", headers=HEADERS).json()
# structure: [{"kitapcikTuru", "lessonId", "dersGrupId", "questionCount", ...}]

# 2) Ham cevapları gönder (her ders için cevaplar uzunluğu questionCount ile eşleşmeli)
# replace GÖVDEDE gönderilir – params={"replace":...} (query) YOK SAYILIR
payload = {
    "replace": False, # True: mevcut sonucun üzerine yaz
    "results": [
        {"okulNumarasi": 1000, "kitapcikTuru": "A", "dersCevapları": [
            {"lessonId": 1, "dersGrupId": 1, "cevaplar": "A" * 40},
            {"lessonId": 2, "dersGrupId": 6, "cevaplar": "B" * 40},
            # ... structure'daki her (lessonId, dersGrupId) için bir satır
        ]},
    ],
}

r = requests.post(f"{BASE_URL}/exams/{exam_id}/results", headers=HEADERS, json=payload)
res = r.json()
print(f"HTTP {r.status_code} – kabul: {res['accepted']}, red: {len(res['rejected'])}")
for row in res["rejected"]:
    print(f" reddedildi idx={row['index']}: {row['reason']}")

# 3) Değerlendirmeyi izle (202 ise jobId döner)
if r.status_code == 202 and res.get("jobId"):
    while True:
        st = requests.get(f"{BASE_URL}/exams/{exam_id}/results/status",

```

```
headers=HEADERS, params={"jobId": res["jobId"]}).json()
print(f"durum: {st['state']}")
if st["state"] in ("Completed", "Failed", "NotFound"):
    break
time.sleep(5)
# 4) state == "Completed" → puanlanmış sonuçları çek
if st["state"] == "Completed":
    results = requests.get(f"{BASE_URL}/exams/{exam_id}/results",
                           headers=HEADERS, params={"MaxResultCount": 100}).json()
```

Ek A: KVKK ve Veri Koruma Notları

- `tcNoHash` tuzsuz SHA-256 olduğundan (TC alanı dar-sayısal) brute-force ile geri çözülebilir; **kişisel veri gibi saklanmalı ve korunmalıdır**. Entegrasyon sözleşmesinde belirtilmelidir.
- `tcNo` açık değeri yalnızca `students:read_pii` scope'u ile döner; bu scope hiçbir hazır pakete dahil değildir ve sözleşmesel onay gerektirir.
- Rehberlik uçları yalnızca **onaylanmış** (veliyle paylaşılabılır) görüşme özetlerini döner.

Ek B: Değişiklik Geçmişi

Sürüm	Tarih	Değişiklik
v1.0	2026-03	İlk sürüm: okuma uçları, yazma uçları, karne oluşturma
v1.1	2026-04	Rate limit header'ları, entegrasyon rehberi, kod örnekleri
v1.3	2026-06	<code>/exams</code> gerçek istatistikler + <code>resultsUpdatedAt</code> / <code>resultStatus</code> / <code>subjectNos</code> ; <code>resultsUpdatedAfter</code> & <code>updatedAfter</code> artımlı senkron filtreleri; <code>/exams/{id}/subjects</code> , <code>/results/lessons</code> , <code>/results/subjects</code> kırılım uçları; <code>tcNoHash</code> / <code>tcNo</code> (read_pii); <code>App.Exam.ResultsPublished</code> webhook'u; sonuçlarda <code>studentNumber</code> / <code>net</code> / <code>updatedAt</code>
v1.4	2026-07	<code>GET /students</code> sunucu tarafı filtreleri: <code>TermId</code> , <code>StudentState</code> (enum, geçersizde 400), <code>ClassroomId</code> , <code>IsActive</code> , <code>ModifiedAfter</code> (artımlı senkron); yanıtta <code>termId</code> / <code>termName</code> / <code>studentState</code> / <code>classroomId</code> / <code>modifiedDate</code> alanları; <code>totalCount</code> artık filtrelenmiş toplamı döndürür; <code>/terms</code> doğru path notu
v1.5	2026-07	Sınav sonucu ingest: <code>GET /exams/{id}/structure</code> (ders/kitapçık/soru sayısı), <code>POST /exams/{id}/results</code> (ham cevap → 202 + <code>jobId</code> , server-side puanlama, <code>replace</code> gövde alanı, <code>rejected[]</code> , tümü redde 422), <code>GET /exams/{id}/results/status</code> (Pending/Running/Completed/Failed + geçersiz jobId'de 200/NotFound); yeni scope <code>exam_results:write</code> (admin/custom); yetkilendirme fail-closed (eşlenmemiş uç → 403); hata tablosuna 404/409/422. Canlı uçtan uca doğrulama: 2026-07-04 (bursa tenant, examId=1518919).

Edesis External API v1.5 · © 2026 Edesis · Bu doküman gizlidir, yalnızca entegrasyon amacıyla kullanılabilir.